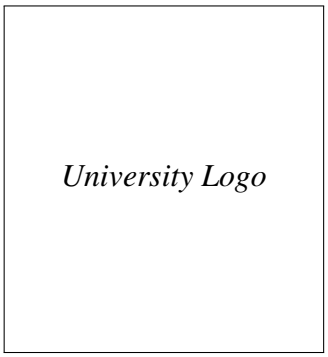


Kingdom of Saudi Arabia

Ministry of Education

University Name

College of Computer Science and Information Technology

Department of Information Technology

Design and Implementation of an AI-Enhanced Multi-Tenant Enterprise Resource Planning System Using Machine Learning for Business Intelligence

A Graduation Project Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Information Technology

Prepared by:

Salah Abdullah Khalil

Supervised by:

[Supervisor Name]

First Semester 1447 / 2025–2026

In the Name of Allah, the Most Gracious,
the Most Merciful

Dedication

I dedicate this work to my beloved parents, whose unwavering support and encouragement have been my greatest source of strength throughout my academic journey. To my family and friends who have stood by me with patience and understanding. And to all those who believe in the power of technology to transform businesses and create a better future.

Acknowledgments

First and foremost, I thank **Allah (SWT)** for granting me the strength, knowledge, and perseverance to complete this graduation project.

I would like to express my sincere gratitude to my supervisor, **[Supervisor Name]**, for the continuous guidance, valuable feedback, and encouragement throughout this research. Your expertise and mentorship have been instrumental in shaping this work.

I am deeply grateful to the **Faculty of Computer Science and Information Technology** at **[University Name]** for providing an enriching academic environment and the resources necessary to undertake this project.

Special thanks to my colleagues and friends who provided constructive criticism, participated in testing, and offered their insights during the development process.

Finally, I extend my heartfelt appreciation to my **family**, especially my parents, for their unconditional love, support, and prayers that have carried me through every challenge.

Abstract

Enterprise Resource Planning (ERP) systems serve as the backbone of modern organizational management by integrating core business processes into a unified platform. However, small and medium enterprises (SMEs) in the Middle East and North Africa (MENA) region face significant barriers to adopting commercial ERP solutions, including prohibitive licensing costs, lack of Arabic language support, and limited customization capabilities. Furthermore, traditional ERP systems largely function as passive record-keeping tools, lacking the predictive and prescriptive intelligence required for proactive business decision-making.

This graduation project presents the design and implementation of an **AI-Enhanced Multi-Tenant ERP System** built on the Django web framework with PostgreSQL database isolation per tenant. The system integrates **eight machine learning models** spanning five business domains: (1) Lead Scoring using Logistic Regression, (2) Late Payment Prediction using Gradient Boosted Trees, (3) Employee Attrition Risk using Random Forest, (4) Expense Anomaly Detection using Isolation Forest, (5) Attendance Pattern Analysis using Z-Score statistics, (6) Demand Forecasting using Facebook Prophet, (7) Cash Flow Projection using rule-based modeling, and (8) KPI Anomaly Detection using statistical process control.

The system was developed following the **Sprint (Agile)** methodology with iterative development cycles. The architecture features over 80 domain models organized into nine functional modules (Finance, HR, CRM, Inventory/WMS, POS, Fixed Assets, Suppliers, Reporting, and AI), a complete REST API, and a real-time AI Insight notification engine. Multi-tenancy is achieved through dynamic PostgreSQL database provisioning via Docker containers or remote server connections, ensuring complete data isolation between organizations.

Experimental evaluation demonstrates strong model performance: Late Payment Prediction achieves an AUC-ROC of **0.9867**, Employee Attrition Prediction achieves **0.9475**, Demand Forecasting achieves an average accuracy of **0.7249** across 80 products using Prophet cross-validation, and the KPI Anomaly Detector successfully identifies statistically significant business metric deviations with calibrated z-score thresholds.

Keywords: ERP System, Machine Learning, Multi-Tenant Architecture, Django, Business Intelligence, Demand Forecasting, Anomaly Detection, Prophet, Gradient Boosting, Agile Development.

Contents

List of Abbreviations **v**

1 Introduction **1**
1.1 Overview . . . 1
1.2 Problem Statement . . . 2
1.3 Project Objectives . . . 2
1.4 Project Scope . . . 3
1.4.1 In Scope . . . 4
1.4.2 Out of Scope . . . 4
1.5 Project Significance . . . 4
1.6 Project Questions . . . 5
1.7 Report Organization . . . 6

2 Literature Review **7**
2.1 Introduction . . . 7
2.2 Enterprise Resource Planning (ERP) Systems . . . 7
2.2.1 Definition and Evolution . . . 7
2.2.2 ERP Architecture Models . . . 8
2.2.3 ERP Adoption in SMEs . . . 8
2.2.4 Open-Source ERP Systems . . . 9
2.3 Multi-Tenant Software Architecture . . . 9
2.3.1 Definition and Models . . . 9
2.3.2 Multi-Tenancy in Django . . . 10
2.4 Machine Learning in Enterprise Systems . . . 11
2.4.1 Overview of ML in Business Applications . . . 11
2.4.2 Lead Scoring with Logistic Regression . . . 11
2.4.3 Payment Risk Prediction with Gradient Boosting . . . 12
2.4.4 Employee Attrition Prediction with Random Forest . . . 12
2.4.5 Anomaly Detection with Isolation Forest . . . 12
2.4.6 Time-Series Forecasting with Prophet . . . 12
2.4.7 Statistical Process Control for KPI Monitoring . . . 13
2.5 Related Work . . . 13

2.5.1 AI-Enhanced ERP Systems . . . 13
2.5.2 Multi-Tenant Django Applications . . . 13
2.5.3 ML for Demand Forecasting in Supply Chain . . . 14
2.6 Gap Analysis . . . 14
2.7 Summary . . . 15

3 Methodology **16**
3.1 Introduction . . . 16
3.2 Development Methodology: Sprint (Agile) . . . 16
3.2.1 Overview of Sprint Methodology . . . 16
3.2.2 Sprint Plan for This Project . . . 17
3.2.3 Sprint Roles . . . 19
3.2.4 Justification for Sprint Methodology . . . 19
3.3 System Architecture . . . 20
3.3.1 High-Level Architecture . . . 20
3.3.2 Multi-Tenant Database Architecture . . . 21
3.3.3 AI/ML Pipeline Architecture . . . 22
3.4 Technology Stack . . . 23
3.5 Data Collection and Preparation . . . 25
3.5.1 Open-Source Datasets . . . 25
3.5.2 Synthetic Data Generation . . . 26
3.6 Machine Learning Model Design . . . 26
3.6.1 Model Selection Criteria . . . 26
3.6.2 Model Specifications . . . 27
3.6.3 Model Evaluation Strategy . . . 28
3.7 Development Tools and Environment . . . 29
3.8 Summary . . . 29

List of Figures

2.1 Evolution of ERP systems from MRP (1970s) to Cloud AI-ERP (2020s) . . 8
2.2 Three levels of multi-tenant data isolation 10

3.1 Sprint (Agile) methodology lifecycle 17
3.2 Gantt chart showing Sprint timeline and dependencies 19
3.3 High-level system architecture showing the three-tier design with multi-tenant database routing 21
3.4 Database-per-tenant architecture with dynamic PostgreSQL provisioning . . 22
3.5 AI/ML pipeline architecture showing the flow from raw data to actionable insights 23

List of Tables

2.1 Comparison of major open-source ERP systems 9
2.2 Literature gap analysis and project contributions 14

3.1 Sprint plan and deliverables 18
3.2 SDLC model comparison and justification 20
3.3 Technology stack 24
3.4 ML model specifications 27
3.5 Development tools and environment 29

List of Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DRF	Django REST Framework
ERP	Enterprise Resource Planning
FK	Foreign Key
GBM	Gradient Boosting Machine
HR	Human Resources
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MAE	Mean Absolute Error
MENA	Middle East and North Africa
ML	Machine Learning
ORM	Object-Relational Mapping
PO	Purchase Order
POS	Point of Sale
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SME	Small and Medium Enterprise
SQL	Structured Query Language
UI	User Interface
UUID	Universally Unique Identifier
WMS	Warehouse Management System

Chapter 1

Introduction

1.1 Overview

Enterprise Resource Planning (ERP) systems have become indispensable tools for organizations seeking to integrate and streamline their core business processes. An ERP system consolidates diverse functional areas—including finance, human resources, customer relationship management, inventory, and procurement—into a single unified platform that facilitates data sharing, eliminates redundancy, and enables informed decision-making (Monk and Wagner, 2013). The global ERP market was valued at approximately USD 54.76 billion in 2022 and is projected to reach USD 123.41 billion by 2030, reflecting the growing recognition of ERP as a strategic asset rather than a mere operational tool (Grand View Research, 2023).

Despite this growth, the adoption of ERP systems among small and medium enterprises (SMEs) in the Middle East and North Africa (MENA) region remains disproportionately low compared to markets in North America and Europe. Research indicates that key barriers include the high total cost of ownership of commercial solutions such as SAP and Oracle, the scarcity of Arabic language support in open-source alternatives, the complexity of deployment and customization, and the limited IT infrastructure in many SMEs (Al-Majali, 2016). These challenges leave a substantial segment of the business landscape relying on disconnected spreadsheets, manual processes, and fragmented software tools that hinder growth and competitiveness.

Concurrently, the rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML) has opened new opportunities for augmenting traditional business systems with predictive and prescriptive capabilities. Modern ML algorithms can analyze historical business data to forecast demand, predict customer payment behavior, detect anomalies in financial transactions, and identify employees at risk of attrition—capabilities that transform an ERP system from a passive record-keeper into a proactive decision-support platform (Davenport and Ronanki, 2018).

This project addresses both challenges simultaneously by designing and implementing an **AI-Enhanced Multi-Tenant ERP System** that is (a) accessible and affordable for SMEs through a multi-tenant Software-as-a-Service (SaaS) architecture, (b) built with Arabic language support from the ground up, and (c) integrated with eight machine learning models that provide actionable business intelligence across five functional domains.

1.2 Problem Statement

The current landscape of ERP adoption among SMEs, particularly in the MENA region, is characterized by several interconnected problems:

1. **High Cost of Commercial ERP Solutions:** Leading ERP systems such as SAP Business One, Oracle NetSuite, and Microsoft Dynamics 365 impose substantial licensing fees, implementation costs, and ongoing maintenance expenses that place them beyond the reach of most SMEs. A Panorama Consulting Group survey found that the average ERP implementation cost exceeds USD 170,000 for small organizations (Panorama Consulting Group, 2023).

2. **Limited Arabic Language Support:** Most open-source ERP platforms (e.g., Odoo Community, ERPNext) provide only partial or community-contributed Arabic translations. Critical modules such as financial reporting, tax compliance, and user interfaces often lack proper right-to-left (RTL) rendering and cultural localization (Al-Kraiji and Al-Saedi, 2020).

3. **Absence of Integrated AI Capabilities:** Traditional ERP systems are designed primarily for recording and reporting historical transactions. They do not incorporate predictive analytics, anomaly detection, or intelligent forecasting within their core workflows. Organizations seeking these capabilities must invest in separate Business Intelligence (BI) tools and custom integrations, increasing both cost and complexity (Lee et al., 2003).

4. **Rigid Single-Tenant Architectures:** Many existing ERP solutions follow a single-tenant deployment model that requires dedicated infrastructure for each organization, driving up hosting costs and administrative overhead. Multi-tenant architectures, which allow multiple organizations to share the same application instance while maintaining complete data isolation, are more cost-effective but technically challenging to implement correctly (Bezemer and Zaidman, 2010).

5. **Lack of Proactive Decision Support:** Business managers frequently make critical decisions—such as inventory reorder quantities, credit terms, employee retention strategies, and cash flow management—based on intuition rather than data-driven insights. The absence of embedded ML models within the ERP workflow means that valuable predictive signals buried in transactional data remain unexploited (Provost and Fawcett, 2013).

1.3 Project Objectives

This project aims to achieve the following objectives:

1. Design and implement a comprehensive, web-based ERP system covering nine functional modules: Finance, Human Resources, Customer Relationship Management (CRM), Inventory and Warehouse Management, Supplier and Procurement, Point of Sale (POS), Fixed Assets, Reporting, and Artificial Intelligence.

2. Implement a robust multi-tenant architecture using database-per-tenant isolation with dynamic PostgreSQL database provisioning via Docker containers and remote server connections.

3. Integrate eight machine learning models into the ERP workflow:

   - *Lead Scoring* — Logistic Regression for CRM conversion prediction
   - *Late Payment Prediction* — Gradient Boosted Trees for accounts receivable risk
   - *Employee Attrition Risk* — Random Forest for HR retention planning
   - *Expense Anomaly Detection* — Isolation Forest for fraud and policy violation identification
   - *Attendance Anomaly Detection* — Z-Score analysis for workforce pattern recognition
   - *Demand Forecasting* — Facebook Prophet for inventory reorder optimization
   - *Cash Flow Projection* — Rule-based modeling for financial planning
   - *KPI Anomaly Detection* — Statistical process control for business metric monitoring

4. Develop an AI Insight Engine that automatically generates actionable alerts and recommendations from ML model outputs and delivers them as real-time notifications to relevant users.

5. Provide bilingual support (English and Arabic) with proper RTL rendering throughout the user interface.

6. Follow the Sprint (Agile) software development methodology to ensure iterative delivery, continuous testing, and stakeholder feedback incorporation.

1.4 Project Scope

The scope of this project encompasses the following:

1.4.1 In Scope

- Full-stack web application development using Django 4.2 (Python) as the backend framework.
- PostgreSQL as the relational database management system for both the main (platform) database and all tenant databases.
- Multi-tenant platform administration: user signup, approval workflow, database provisioning, and role-based access control (Owner, Admin, User, Read-Only).
- Nine ERP functional modules with complete CRUD operations, business logic, and reporting.
- Eight ML models with training pipelines, model versioning, prediction logging, and accuracy evaluation.
- RESTful API layer using Django REST Framework (DRF) for mobile and third-party integration.
- AI Insight notification system with severity levels (Info, Warning, Critical) and automatic deduplication.
- Asynchronous task processing using Celery with Redis as the message broker.
- Data seeding utilities for testing and demonstration.

1.4.2 Out of Scope

- Native mobile application development (the system is accessible via mobile browsers and a REST API is provided for future native app development).
- Integration with external accounting authorities or government e-invoicing APIs (e.g., ZATCA in Saudi Arabia).
- Deep learning models (CNN, RNN, Transformers) for natural language processing or image recognition—the AI scope is limited to classical ML and time-series forecasting.
- Load testing and performance benchmarking for production-scale deployments.

1.5 Project Significance

This project makes several contributions:

1. **Practical Contribution:** It delivers a fully functional, production-ready ERP system that SMEs in the MENA region can deploy without the licensing costs associated with commercial alternatives. The multi-tenant SaaS model further reduces per-organization costs by sharing infrastructure.

2. **Technical Innovation:** To the best of our knowledge, this is among the first open-source ERP implementations that embeds eight ML models directly within the transactional workflow—not as an external BI layer, but as integral components that train on the organization's own data, generate predictions in real time, and write actionable insights back into the ERP.

3. **Academic Contribution:** The project demonstrates the practical application of multiple ML algorithms (Logistic Regression, Gradient Boosting, Random Forest, Isolation Forest, Prophet, Z-Score) to real-world ERP datasets, providing empirical performance metrics (AUC-ROC, MAE, F1-Score) that contribute to the body of knowledge on applied machine learning in business systems.

4. **Bilingual Accessibility:** By incorporating Arabic names, labels, and RTL support at the model level (dual-language fields throughout all 80+ models), the system addresses a persistent localization gap in open-source ERP solutions.

1.6 Project Questions

This project seeks to answer the following research questions:

RQ1: How can a multi-tenant ERP system with database-per-tenant isolation be effectively designed and implemented using Django and PostgreSQL to achieve both data security and cost efficiency?

RQ2: To what extent can machine learning models trained on ERP transactional data provide accurate and actionable predictions for business processes such as demand forecasting, payment risk assessment, and employee attrition?

RQ3: What software architecture and design patterns are required to seamlessly integrate ML model training, prediction, and insight generation into the standard ERP operational workflow?

RQ4: How does the Sprint (Agile) development methodology facilitate the iterative development of a complex, multi-module ERP system with integrated AI capabilities?

1.7 Report Organization

The remainder of this report is organized as follows:

- **Chapter 2 — Literature Review:** Provides a comprehensive review of existing ERP systems, multi-tenant architectures, and the application of machine learning in enterprise systems. It examines related work and identifies the gaps that this project addresses.
- **Chapter 3 — Methodology:** Describes the Sprint (Agile) development methodology adopted, the system architecture, the technology stack, and the ML model design and evaluation approach.
- **Chapter 4 — System Analysis and Design:** Presents the system requirements (functional and non-functional), use case diagrams, class diagrams, database schema design, and the AI/ML pipeline architecture.
- **Chapter 5 — Implementation and Testing:** Details the implementation of key system components, code structure, ML model training results, and testing procedures.
- **Chapter 6 — Results and Discussion:** Analyzes the ML model performance metrics, system usability, and discusses the findings in the context of the research questions.
- **Chapter 7 — Conclusion and Future Work:** Summarizes the contributions, acknowledges limitations, and proposes directions for future enhancement.

Chapter 2

Literature Review

2.1 Introduction

This chapter provides a comprehensive review of the theoretical foundations and existing work relevant to the three core pillars of this project: Enterprise Resource Planning systems, multi-tenant software architectures, and the application of machine learning in business systems. The review establishes the academic context, identifies gaps in current solutions, and justifies the design decisions adopted in this project.

2.2 Enterprise Resource Planning (ERP) Systems

2.2.1 Definition and Evolution

Enterprise Resource Planning (ERP) is defined as an integrated software system that manages and automates core business processes across an organization through a shared database and standardized workflows (Monk and Wagner, 2013). The evolution of ERP systems can be traced through three generations:

1. **Material Requirements Planning (MRP), 1970s:** Focused exclusively on manufacturing—calculating raw material needs based on production schedules and bills of materials. IBM's COPICS and SAP R/1 were early examples (Jacobs and Weston, 2007).

2. **MRP II and Early ERP, 1980s–1990s:** Extended MRP to include finance, HR, and distribution. SAP R/3, introduced in 1992, pioneered the client-server ERP model with real-time integration across modules (Jacobs and Weston, 2007). This era established the "ERP" terminology and saw rapid adoption by large enterprises.

3. **Modern Cloud ERP, 2010s–Present:** The shift to web-based, cloud-hosted architectures has democratized ERP access. Solutions like SAP S/4HANA Cloud, Oracle NetSuite, and open-source platforms such as Odoo and ERPNext enable SaaS deployment models that reduce upfront investment (Panorama Consulting Group, 2023).

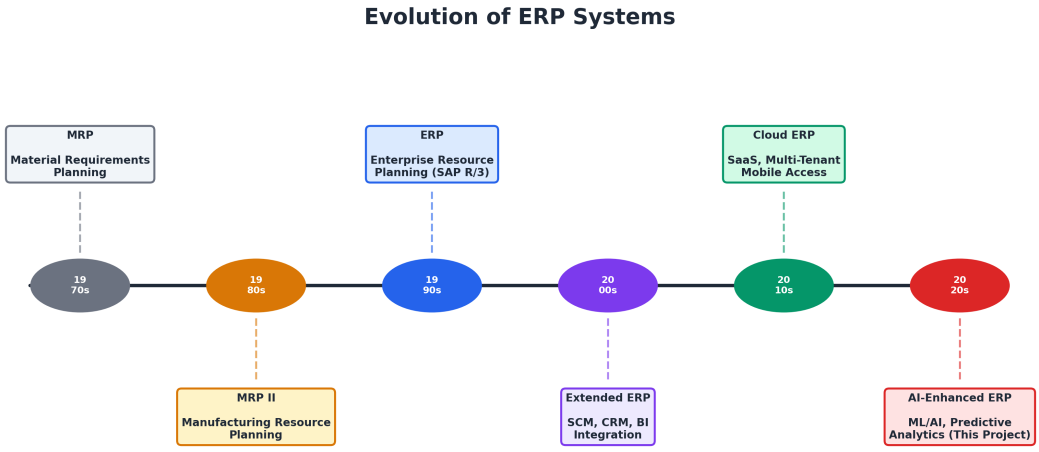


Figure 2.1: Evolution of ERP systems from MRP (1970s) to Cloud AI-ERP (2020s)

2.2.2 ERP Architecture Models

Modern ERP systems are typically deployed using one of three architectural models:

- **Monolithic Architecture:** All modules are packaged as a single deployable unit sharing one database. Traditional SAP R/3 and Oracle E-Business Suite follow this model. While simple to deploy, monolithic architectures are difficult to scale and customize (Bezemer and Zaidman, 2010).
- **Service-Oriented Architecture (SOA):** Business functions are decomposed into loosely coupled services that communicate via standardized protocols. Modern SAP S/4HANA adopts elements of SOA to enable extensibility (Krafzig et al., 2005).
- **Microservices Architecture:** An evolution of SOA where each service is independently deployable and scalable. While offering maximum flexibility, this model introduces operational complexity that makes it less suitable for SME-focused projects (Newman, 2015).

This project adopts a **modular monolith** architecture—a single Django application with clearly separated modules (apps) that share a common codebase but maintain logical boundaries. This balances simplicity with maintainability and is well-suited for the project scope.

2.2.3 ERP Adoption in SMEs

Research by Al-Majali (2016) identifies organizational factors (top management support, organizational readiness), technological factors (system complexity, compatibility), and environmental factors (competitive pressure, vendor support) as determinants of ERP adoption success in Jordanian SMEs. Their findings highlight that *perceived cost* and *organizational readiness* are the strongest predictors of adoption intent.

A study by Panorama Consulting Group (2023) found that among organizations implementing ERP, 50% reported exceeding their initial budget, and the average implementation timeline for SMEs was 14.3 months. These statistics underscore the need for lightweight, rapid-deployment ERP solutions—a gap this project addresses through its Django-based architecture and Sprint development methodology.

2.2.4 Open-Source ERP Systems

Several open-source ERP platforms have emerged as alternatives to commercial solutions:

Table 2.1: Comparison of major open-source ERP systems

Feature	**Odoo CE**	**ERPNext**	**Dolibarr**	**This Project**
Language	Python	Python/JS	PHP	Python (Django)
Database	PostgreSQL	MariaDB	MySQL/PgSQL	PostgreSQL
Multi-Tenant	Limited	No	No	**Yes (DB-per-tenant)**
Arabic Support	Partial	Partial	Minimal	**Full (dual fields)**
Integrated ML/AI	No	No	No	**8 ML models**
REST API	Yes	Yes	Limited	Yes (DRF)
WMS	Enterprise only	Basic	No	**Full WMS**
POS	Enterprise only	Basic	No	**Yes**
License	LGPL v3	GPL v3	GPL v3	–

As Table 2.1 illustrates, none of the existing open-source ERP systems combine native multi-tenant isolation, comprehensive Arabic support, and integrated machine learning capabilities—the three distinguishing features of this project.

2.3 Multi-Tenant Software Architecture

2.3.1 Definition and Models

Multi-tenancy is a software architecture principle in which a single instance of a software application serves multiple customers (tenants), each of whom perceives the application as if it were running exclusively for them (Bezemer and Zaidman, 2010). Three principal models of multi-tenancy exist:

1. **Shared Database, Shared Schema:** All tenants share the same database and tables, with a `tenant_id` column used to filter data. This is the most resource-efficient

model but offers the weakest data isolation and complicates compliance with data sovereignty regulations (Chong and Carraro, 2006).

2. **Shared Database, Separate Schema:** Each tenant has its own database schema (namespace) within a shared database instance. This provides moderate isolation with lower overhead than full database separation. PostgreSQL's native schema support makes this model practical (Bezemer and Zaidman, 2010).

3. **Separate Database (Database-per-Tenant):** Each tenant is assigned a dedicated database instance. This model provides the strongest isolation—tenant data is physically separated, simplifying backup, migration, and regulatory compliance—at the cost of higher resource consumption (Bezemer and Zaidman, 2010).

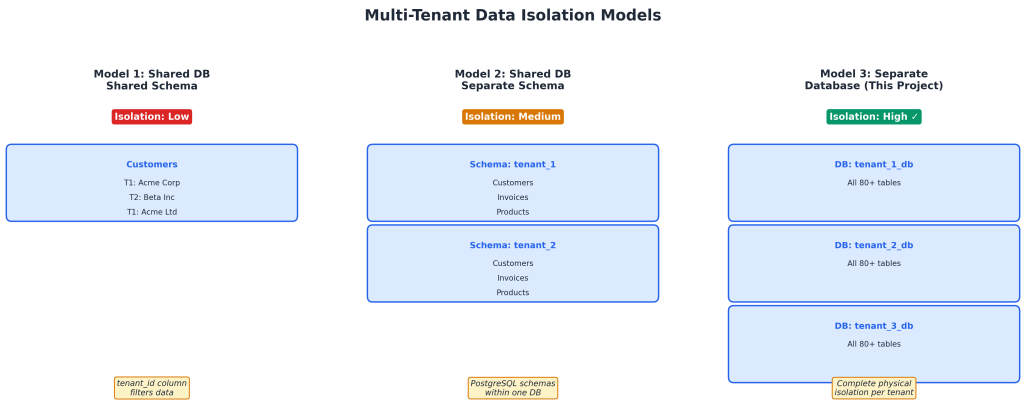


Figure 2.2: Three levels of multi-tenant data isolation

This project implements the **database-per-tenant** model, which provides the highest level of data isolation. Each tenant organization receives a dedicated PostgreSQL database that is dynamically provisioned via Docker containers or remote server connections. Django's database routing mechanism (`DATABASE_ROUTERS`) is used to direct queries to the appropriate tenant database based on the authenticated user's session context.

2.3.2 Multi-Tenancy in Django

Django's built-in support for multiple databases, combined with custom database routers, makes it well-suited for implementing multi-tenant architectures. The key components in this project's implementation are:

- `TenantDatabase` model: Stores connection parameters for each tenant's PostgreSQL database.
- `DatabaseAssignment` model: Maps users to tenant databases with role-based permissions.

- `add_tenant_database()`: Dynamically registers tenant database configurations in Django's `DATABASES` setting at runtime.
- `TenantDatabaseContext`: A context manager that sets the active database alias for the current thread, ensuring all ORM queries route to the correct tenant database.
- Custom middleware: Resolves the authenticated user's tenant database on each request and activates the appropriate database context.

2.4 Machine Learning in Enterprise Systems

2.4.1 Overview of ML in Business Applications

The application of machine learning to business processes has accelerated significantly with the availability of large transactional datasets, affordable computing power, and mature ML frameworks. Davenport and Ronanki (2018) identify three levels of AI integration in business systems:

1. **Process Automation:** Using ML for routine tasks such as document classification, data entry validation, and alert generation.
2. **Cognitive Insight:** Applying ML algorithms to detect patterns in large datasets—such as customer segmentation, fraud detection, and demand forecasting—that would be impractical for human analysts.
3. **Cognitive Engagement:** Deploying conversational AI and recommendation systems that interact directly with users, such as chatbots and personalized marketing engines.

This project primarily operates at the **Cognitive Insight** level, embedding ML models that analyze transactional data and surface actionable insights to business users through the AI Insight Engine.

2.4.2 Lead Scoring with Logistic Regression

Lead scoring is the process of assigning a numerical value to each sales lead to predict the likelihood of conversion. Provost and Fawcett (2013) describe logistic regression as particularly well-suited for lead scoring because it provides calibrated probability estimates (not just classifications), is inherently interpretable, and performs well with moderate-sized datasets.

In this project, the Lead Scoring model uses features including estimated deal value, lead age, activity count, lead source, and customer contact completeness. The model is trained on historical won/lost leads and outputs a conversion probability between 0 and 1.

2.4.3 Payment Risk Prediction with Gradient Boosting

Predicting whether an invoice will be paid late is a critical capability for accounts receivable management. Chen and Guestrin (2016) demonstrate that gradient boosted decision trees consistently outperform other classifiers on structured/tabular data tasks, making them ideal for invoice-level payment risk prediction.

The project's Late Payment Predictor uses 10 engineered features, including invoice amount, days to due date, customer historical payment delay, and customer type. Leave-one-out temporal statistics are used to prevent data leakage—a common pitfall identified by Provost and Fawcett (2013) in churn and payment prediction models.

2.4.4 Employee Attrition Prediction with Random Forest

Employee turnover is costly, with the Society for Human Resource Management estimating replacement costs at 50–200% of annual salary (Society for Human Resource Management, 2022). Breiman (2001) introduced Random Forest—an ensemble of decision trees trained on bootstrap samples—which has become a standard approach for employee attrition prediction due to its robustness to noisy features and native feature importance ranking.

This project's Attrition Predictor uses 10 features including tenure, salary relative to department average, leave frequency, attendance rate, and overtime hours. Class imbalance (typically fewer departures than retentions) is addressed using the `class_weight='balanced'` parameter.

2.4.5 Anomaly Detection with Isolation Forest

Liu et al. (2008) proposed the Isolation Forest algorithm based on the principle that anomalies are "few and different"—they require fewer random partitions to isolate than normal observations. This makes Isolation Forest computationally efficient and effective for unsupervised anomaly detection in expense claim analysis, where labeled fraud data is typically unavailable.

The project's Expense Anomaly Detector applies Isolation Forest with a `contamination` parameter of 0.03 (expecting 3% of expenses to be anomalous) to features including expense amount, category, day of week, and month.

2.4.6 Time-Series Forecasting with Prophet

Facebook Prophet, introduced by Taylor and Letham (2018), is an additive regression model designed for business time-series data with strong seasonal patterns and missing data points. Prophet decomposes the forecast into trend, seasonality (weekly, yearly, custom), and holiday components using a Bayesian framework.

The project uses Prophet for per-product demand forecasting, with custom monthly seasonality ($fourier_order = 3$) and calibrated changepoint detection ($changepoint_prior_scale = 0.15$). Model accuracy is evaluated using Prophet's built-in cross-validation framework, achieving an average accuracy of 0.7249 across 80 product-warehouse combinations.

2.4.7 Statistical Process Control for KPI Monitoring

Z-score-based anomaly detection is a classical statistical quality control technique introduced by Shewhart (1931) and widely used in manufacturing as Shewhart control charts. When applied to business KPIs, the method flags values that deviate more than a specified number of standard deviations from the historical mean.

The project's KPI Anomaly Detector monitors five daily metrics (revenue, expenses, new leads, overdue invoices, inventory movements) against 90-day rolling baselines, generating WARNING alerts for deviations exceeding 2σ and CRITICAL alerts for deviations exceeding 3σ.

2.5 Related Work

2.5.1 AI-Enhanced ERP Systems

Lee et al. (2003) survey the integration of AI technologies into ERP systems and identify three implementation strategies: (1) embedded AI within the ERP platform itself, (2) AI as an external service integrated via APIs, and (3) AI as a middleware layer between the ERP and a BI platform. They conclude that the embedded approach provides the tightest integration and lowest latency but requires the deepest technical expertise. This project follows the embedded approach.

SAP introduced "SAP Business AI" in 2023, incorporating predictive analytics and natural language processing into S/4HANA Cloud. However, these capabilities are available only in the Enterprise tier, which carries substantial licensing costs. Oracle Fusion Cloud ERP similarly offers ML-driven anomaly detection and forecasting features in its premium offerings. No open-source ERP system currently provides comparable embedded ML capabilities.

2.5.2 Multi-Tenant Django Applications

The `django-tenants` library implements PostgreSQL schema-based multi-tenancy for Django, providing automatic schema routing via the `search_path` mechanism. While this approach is well-tested, it does not support database-per-tenant isolation, which this project requires for maximum data separation. The `django-db-multitenant` library offers a middleware-based approach but is limited to schema-based or connection-string routing.

This project implements a custom multi-tenant framework with dynamic database registration, Docker-based provisioning, and a user approval workflow—features not provided by existing libraries.

2.5.3 ML for Demand Forecasting in Supply Chain

Fildes et al. (2022) provide a comprehensive review of ML approaches for retail demand forecasting, comparing classical time-series methods (ARIMA, Exponential Smoothing) with machine learning methods (Random Forest, Gradient Boosting, Neural Networks) and hybrid approaches. They find that Prophet performs competitively on datasets with strong seasonality and limited history, making it well-suited for SME inventory data where per-product histories may span only 6–12 months.

2.6 Gap Analysis

The literature review reveals the following gaps that this project addresses:

Table 2.2: Literature gap analysis and project contributions

Gap Identified	**Supporting Literature**	**Project Contribution**
No open-source ERP with embedded ML	Lee et al. (2003); SAP/Oracle features are premium-only	8 ML models embedded in a free, open-source ERP
Limited multi-tenant options in Django	django-tenants supports schema-only; django-db-multitenant lacks provisioning	Custom DB-per-tenant with Docker provisioning and role-based access
Poor Arabic support in open-source ERP	Al-Kraiji and Al-Saedi (2020); Odoo/ERPNext have partial translations	Dual-language fields (`name`/`name_ar`) in 80+ models
ML models disconnected from ERP workflow	Most ML-ERP integrations use external BI tools	Inline ML training, prediction logging, and AI Insight notifications
Lack of cross-domain ML in single ERP	Studies focus on single domains (finance OR HR OR supply chain)	Models span CRM, Finance, HR, Inventory, and Operations

2.7 Summary

This chapter reviewed the evolution and current state of ERP systems, multi-tenant software architectures, and the application of machine learning in enterprise settings. The review established that while commercial ERP vendors have begun incorporating AI capabilities, these features remain inaccessible to SMEs due to cost and complexity. Open-source ERP alternatives lack both native multi-tenancy and integrated ML. This project bridges these gaps by combining a production-quality multi-tenant ERP with eight embedded ML models, delivered through an Agile Sprint development process that is detailed in the following chapter.

Chapter 3

Methodology

3.1 Introduction

This chapter presents the development methodology, system architecture, technology stack, data collection approach, and machine learning pipeline design adopted for this project. The Sprint (Agile) methodology was selected as the software development lifecycle (SDLC) model to accommodate the project's complexity, evolving requirements, and the iterative nature of ML model development.

3.2 Development Methodology: Sprint (Agile)

3.2.1 Overview of Sprint Methodology

The Sprint methodology, as defined in the Scrum framework (Schwaber and Sutherland, 2020), organizes software development into fixed-length iterations called **Sprints**, typically lasting 1–4 weeks. Each Sprint follows a structured cycle: planning, development, testing, review, and retrospective. This iterative approach is particularly well-suited for projects with complex, multi-module architectures and evolving requirements—both characteristics of this ERP system.

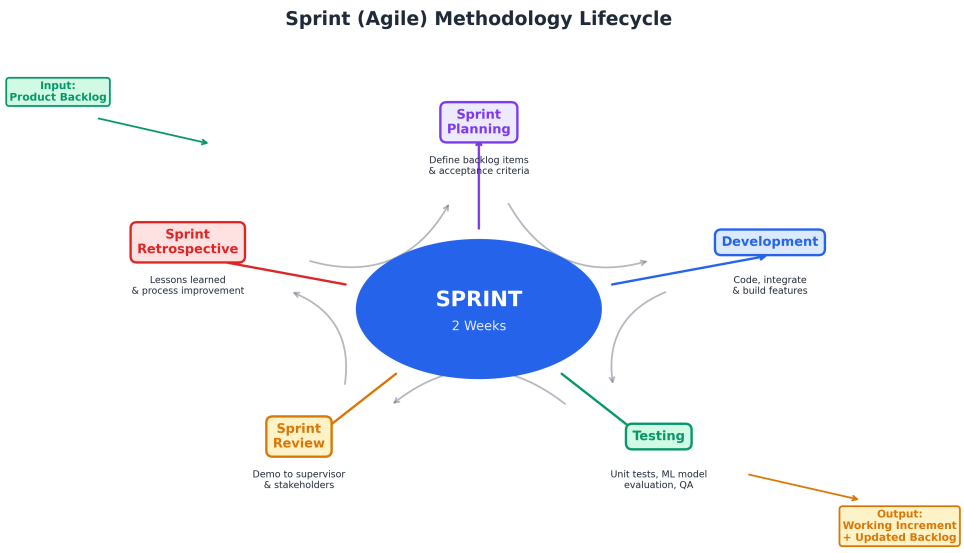


Figure 3.1: Sprint (Agile) methodology lifecycle

The key principles of the Sprint methodology applied to this project include:

- **Iterative Delivery:** Each Sprint produces a working increment of the system, allowing stakeholders to review functional modules progressively rather than waiting for a final monolithic delivery.
- **Adaptive Planning:** Sprint planning sessions at the beginning of each iteration allow the team to reprioritize tasks based on feedback, discovered dependencies, and technical challenges.
- **Continuous Integration and Testing:** Code is tested at the end of each Sprint, including unit tests for ML model accuracy and integration tests for API endpoints.
- **Feedback-Driven Improvement:** Sprint retrospectives identify process improvements, while Sprint reviews validate that delivered features meet acceptance criteria.

3.2.2 Sprint Plan for This Project

The project was organized into **seven Sprints** over the development period. Table 3.1 summarizes the Sprint plan:

Table 3.1: Sprint plan and deliverables

Sprint	Duration	Focus Area	Deliverables
1	2 weeks	Project Setup & Multi-Tenant Architecture	Django project scaffold, TenantUser model, PostgresServer model, TenantDatabase model, database routing, user approval workflow
2	2 weeks	Core ERP Modules (Finance & HR)	Company, Currency, Department, Employee, Attendance, Payroll, Invoice, Payment, Expense, Chart of Accounts
3	2 weeks	CRM, Supplier & Procurement	Lead, LeadActivity, Campaign, Customer, Supplier, PurchaseOrder, SupplierInvoice, SupplierPayment
4	2 weeks	Inventory/WMS & POS	Warehouse, zones/bins, StockMovement, GoodsReceipt, StockTransfer, CycleCount, PickList, ShipmentNote, ReorderRule, POS terminal/session/order
5	2 weeks	ML Model Development	AI model registry, 8 ML service modules, training pipeline, prediction logging, AI Insight engine
6	2 weeks	Data Integration & ML Tuning	Dataset import commands, synthetic data seeding, Prophet hyperparameter tuning, KPI calibration, model evaluation
7	2 weeks	Testing, Documentation & Deployment	Integration testing, API documentation, deployment configuration (Gunicorn + Docker), thesis writing

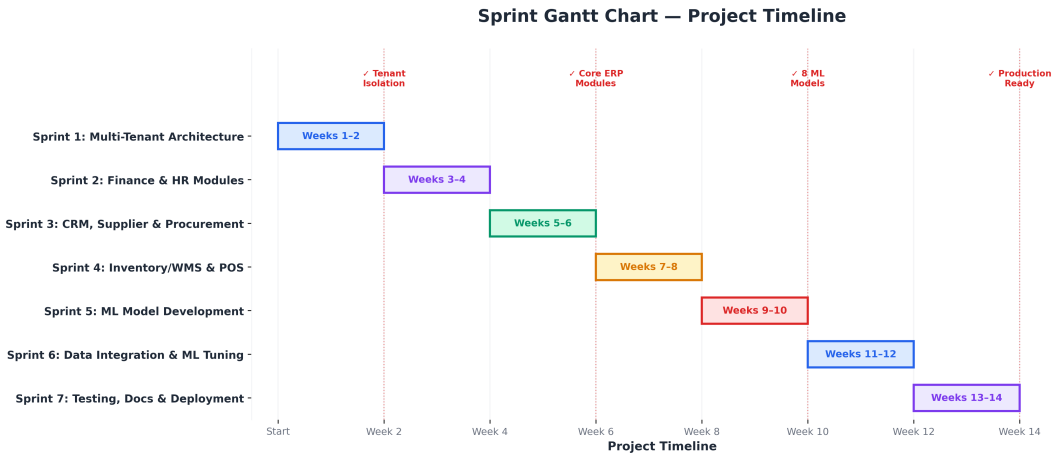


Figure 3.2: Gantt chart showing Sprint timeline and dependencies

3.2.3 Sprint Roles

Although this is an individual graduation project, the Sprint methodology roles were adapted as follows:

- **Product Owner:** The project supervisor, who reviewed Sprint deliverables and provided feedback on priorities and acceptance criteria.
- **Scrum Master / Developer:** The student (Salah Abdullah Khalil), responsible for Sprint planning, development, testing, and retrospective documentation.
- **Stakeholders:** The academic committee and potential end users who provided domain feedback during the CRM, Finance, and HR module development.

3.2.4 Justification for Sprint Methodology

The Sprint methodology was chosen over alternative SDLC models for the following reasons:

Table 3.2: SDLC model comparison and justification

SDLC Model	Strengths	Limitation for This Project
Waterfall	Clear phases, thorough documentation	Does not accommodate changing requirements; ML models require iterative tuning
V-Model	Strong testing emphasis	Sequential nature delays integration testing
RAD	Rapid prototyping	Insufficient emphasis on architecture for a system with 80+ models
Sprint (Agile)	Iterative delivery, accommodates ML experimentation, continuous feedback	Requires discipline in documentation (addressed through Sprint reviews)

3.3 System Architecture

3.3.1 High-Level Architecture

The system follows a three-tier architecture: Presentation Layer (Django templates + REST API), Application Layer (Django views, services, and ML pipelines), and Data Layer (PostgreSQL databases with multi-tenant routing).

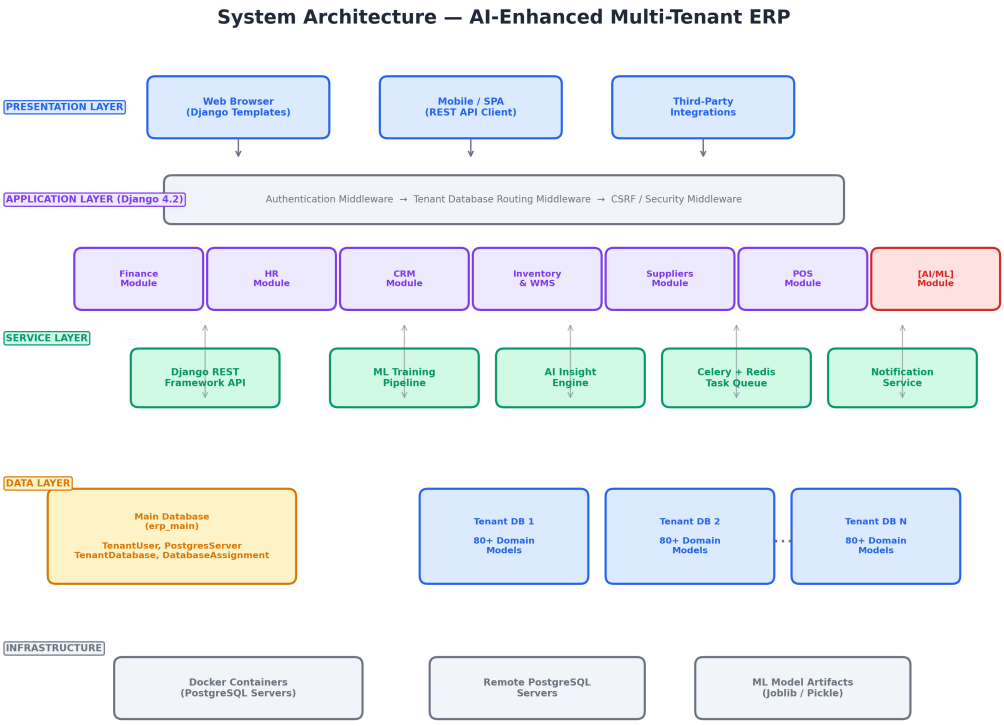


Figure 3.3: High-level system architecture showing the three-tier design with multi-tenant database routing

3.3.2 Multi-Tenant Database Architecture

The multi-tenant architecture is implemented using the database-per-tenant isolation model. The platform maintains a **main database** (`erp_main`) that stores user accounts, server configurations, and tenant metadata. Each approved tenant organization receives a dedicated PostgreSQL database provisioned on a selected server (Docker container or remote server).

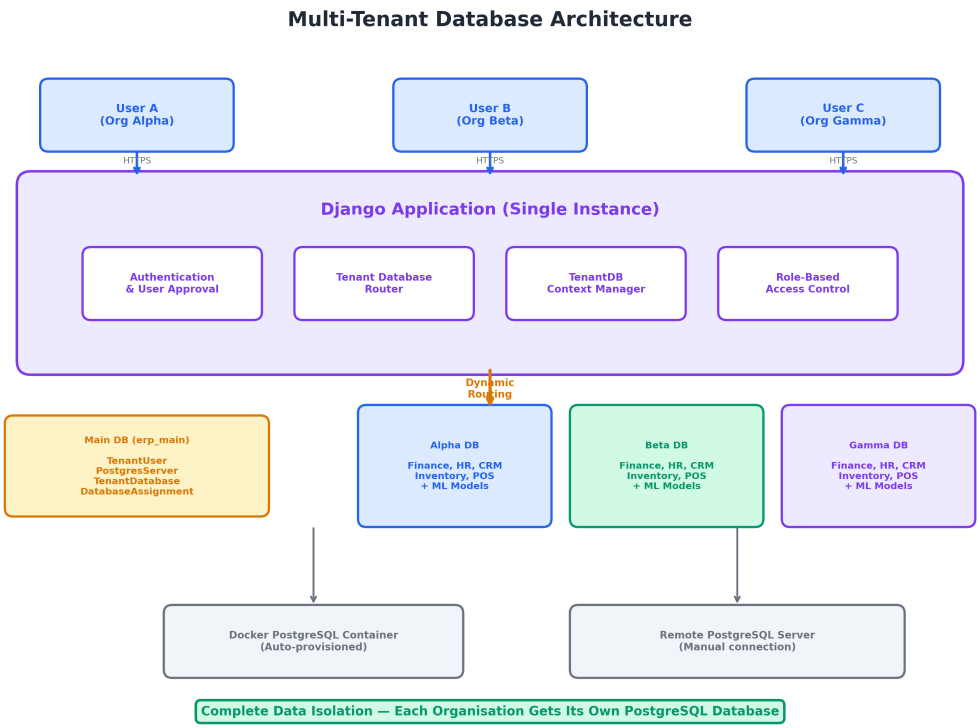


Figure 3.4: Database-per-tenant architecture with dynamic PostgreSQL provisioning

The database routing mechanism works as follows:

1. The user logs in and the authentication middleware identifies their `DatabaseAssignment`.
2. The `add_tenant_database()` function dynamically registers the tenant's database connection parameters in Django's `DATABASES` dictionary.
3. The `TenantDatabaseContext` context manager sets the active database alias for the current request thread.
4. All subsequent ORM queries are automatically routed to the tenant's database via the custom `DATABASE_ROUTERS` configuration.
5. When the request completes, the context manager resets the routing.

3.3.3 AI/ML Pipeline Architecture

The ML pipeline is integrated into the ERP as a self-contained module (`erp.ai`) with the following components:

- **Model Registry** (`MLModel`): Tracks model versions, accuracy scores, training parameters, and artefact file paths. Each model is scoped to a specific company (tenant).

- **Service Layer** (`erp.ai.services`): Contains the implementation of each ML model as a Python class. Each service class follows a consistent interface: `train()`, `predict()`, and an optional `score_all()` batch method.
- **Training Pipeline** (`train_models` management command): An orchestrator that sequentially trains all selected models, evaluates their performance, and generates a JSON training report.
- **Prediction Logging** (`PredictionLog`): An immutable audit trail of every prediction made, enabling future retraining on actual outcomes.
- **AI Insight Engine** (`AIInsight` + `create_or_update_insight`): Converts ML model outputs into human-readable alerts with severity levels, recommendations, and action URLs. Includes automatic deduplication via a configurable cooldown window.

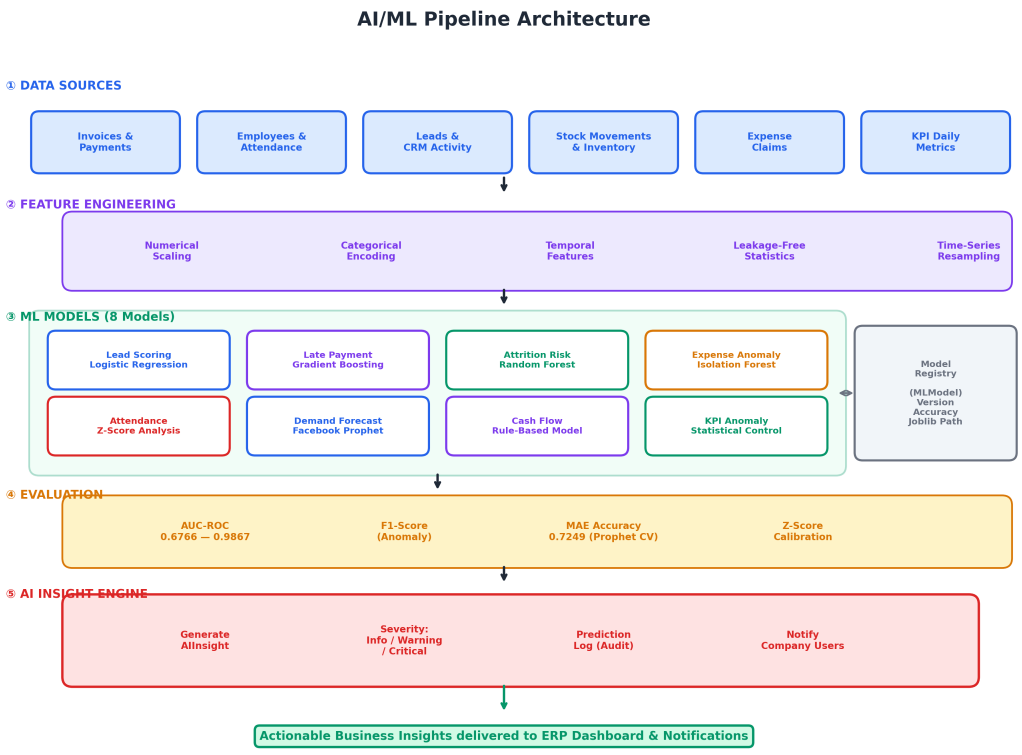


Figure 3.5: AI/ML pipeline architecture showing the flow from raw data to actionable insights

3.4 Technology Stack

Table 3.3 summarizes the technologies used in this project:

Table 3.3: Technology stack

Layer	Technology	Purpose
Backend Framework	Django 4.2	Web framework, ORM, URL routing, template engine
Programming Language	Python 3.12	Primary development language
Database	PostgreSQL 12+	Relational database for main and tenant databases
REST API	Django REST Framework 3.14	RESTful API layer for mobile/third-party integration
ML / Data Science	scikit-learn 1.4, pandas 2.0, numpy 1.26	Classical ML algorithms and data processing
Time-Series Forecasting	Facebook Prophet 1.1.5	Additive regression for demand forecasting
LLM Integration	OpenAI API (GPT-4o-mini)	AI assistant and natural language analysis
Task Queue	Celery 5.3 + Redis 5.0	Asynchronous ML

3.5 Data Collection and Preparation

The ML models in this project are trained on a combination of real-world open-source datasets and synthetic domain-specific data generated to fill coverage gaps:

3.5.1 Open-Source Datasets

1. **Olist Brazilian E-Commerce Dataset** (Olist, 2018): A public dataset from Kaggle containing over 100,000 real e-commerce orders from Brazilian marketplaces (2016–2018). This dataset provides:

   - 99,441 orders with order dates, delivery dates, and payment information
   - 32,216 products across 73 categories
   - 96,096 customers with geolocation data
   - 3,095 sellers

   The Olist dataset was used to populate the Customers, Products, Invoices, Orders, and Payments modules.

2. **IBM HR Employee Attrition Dataset** (IBM, 2017): A synthetic but statistically realistic dataset of 1,470 employee records with 35 attributes including salary, department, overtime hours, and attrition labels. This dataset trains the Employee Attrition Prediction model.

3. **Supplier Procurement Dataset**: A custom-generated dataset of 50 suppliers and 873 purchase orders with realistic procurement cycles, used to populate the Supplier and Purchase Order modules.

4. **Attendance Timeseries Dataset**: A custom-generated dataset of employee attendance records spanning 12 months with check-in/check-out times, hours worked, late arrival flags, and overtime hours.

5. **Expense Claims Dataset**: A custom-generated dataset of 6,278 approved expense claims with embedded anomalies (labeled `is_anomaly` column) for Isolation Forest evaluation.

6. **CRM Leads Dataset**: A custom-generated dataset of 4,062 leads with source, estimated value, activities, and won/lost outcomes.

3.5.2 Synthetic Data Generation

For modules without corresponding real-world datasets—particularly Inventory/WMS and KPI monitoring—a data seeding utility (`seed_ml_operational_data`) was developed. This management command generates:

- **Stock Movements:** 365 days of daily demand history for 80 products with weekly and monthly seasonality, product diversity profiles (fast/moderate/slow movers), weekend dips, growth trends, and random noise.
- **Reorder Rules:** 80 product-warehouse rule configurations with randomized lead times, safety stock levels, and minimum order quantities.
- **Supplier Invoices:** 220 pending supplier invoices with due dates spanning the next 90 days for Cash Flow projection.
- **KPI Spike Injection:** Calibrated inventory movement spikes targeting z-score $\approx 3.5\sigma$ above the 90-day baseline for KPI Anomaly Detection testing.

3.6 Machine Learning Model Design

3.6.1 Model Selection Criteria

The ML models were selected based on four criteria:

1. **Interpretability:** Business users must understand why a prediction was made. Models with native feature importance (Random Forest, Gradient Boosting) and calibrated probabilities (Logistic Regression) were preferred.
2. **Data Efficiency:** SME datasets are typically small (hundreds to low thousands of records). Models that perform well with limited data (Prophet, Z-Score, Isolation Forest) were favored.
3. **Operational Simplicity:** Models must train in seconds to minutes on a standard server, serialize as compact files (joblib), and produce predictions with sub-second latency.
4. **Task Appropriateness:** Each algorithm was matched to its problem type—classification (Logistic Regression, GBM, Random Forest), unsupervised anomaly detection (Isolation Forest, Z-Score), time-series forecasting (Prophet), and rule-based projection (Cash Flow).

3.6.2 Model Specifications

Table 3.4 summarizes the eight ML models implemented in this project:

Table 3.4: ML model specifications

Model	Domain	Algorithm	Features	Evaluation Metric	Min. Samples
Lead Scoring	CRM	Logistic Regression (C=1.0)	7 features (value, age, activities, source, contact info)	AUC-ROC	30
Late Payment	Finance	Gradient Boosting (300 trees, depth=5, lr=0.08)	10 features (amount, days-to-due, customer history)	AUC-ROC	30
Attrition	HR	Random Forest (300 trees, depth=8, balanced)	10 features (tenure, salary, attendance, overtime)	AUC-ROC	30
Expense Anomaly	Finance	Isolation Forest (300 trees, contamination=3%)	4 features (amount, month, weekday, category)	F1-Score vs. ground truth	30
Attendance Anomaly	HR	Z-Score (threshold=2.0)	2 metrics (avg hours, late arrival ratio)	Anomaly count	1
Demand Forecast	Inventory	Prophet (additive, weekly + monthly seasonality)	Daily demand history per product/warehouse	MAE-based accuracy via cross-validation	30

Table 3.4: ML model specifications (continued)

Model	Domain	Algorithm	Features	Evaluation Metric	Min. Samples
Cash Flow	Finance	Rule-based (30/60/90-day projection)	Outstanding invoices + pending payables + open POs	Net cash position	0
KPI Anomaly	Operations	Z-Score (threshold $\geq 2\sigma$ warning, $\geq 3\sigma$ critical)	5 daily KPIs (revenue, expenses, leads, overdue invoices, inventory movements)	Alert accuracy	14

3.6.3 Model Evaluation Strategy

Each model is evaluated using appropriate metrics for its task type:

- **Classifiers** (Lead Scoring, Late Payment, Attrition): AUC-ROC on a stratified 80/20 train/test split. AUC-ROC is preferred over accuracy because the datasets exhibit class imbalance.
- **Anomaly Detectors** (Expense Anomaly): Precision, Recall, and F1-Score computed against a ground-truth `is_anomaly` label in the expense dataset.
- **Time-Series Forecasts** (Demand): MAE-based accuracy ($1 - \frac{MAE}{\bar{y}}$) computed via Prophet's built-in cross-validation with expanding windows.
- **Statistical Detectors** (Attendance Anomaly, KPI Anomaly): Qualitative evaluation based on the number of alerts generated and their clinical reasonableness (e.g., z-score magnitude aligning with injected anomalies).
- **Cash Flow**: Validation against known payables and receivables in the database—checking that inflows equal outstanding invoice balances and outflows match pending supplier invoices.

3.7 Development Tools and Environment

Table 3.5: Development tools and environment

Category	Tool	Purpose
IDE	Visual Studio Code	Primary code editor with Python and Django extensions
AI Pair Programming	GitHub Copilot (Claude)	AI-assisted code generation and debugging
Version Control	Git + GitHub	Source code management and collaboration
Database Client	pgAdmin 4	PostgreSQL database management and querying
API Testing	Django REST Framework browsable API	Interactive API testing and documentation
Virtual Environment	Python venv	Dependency isolation
Operating System	Windows 11	Development machine OS
Browser	Google Chrome DevTools	Frontend debugging and responsive design testing

3.8 Summary

This chapter detailed the Sprint (Agile) methodology adopted for the project, breaking the development into seven two-week Sprints covering platform architecture through ML tuning and deployment. The three-tier system architecture with database-per-tenant multi-tenancy was described, along with the AI/ML pipeline that trains, evaluates, logs, and surfaces model outputs as actionable business insights. The technology stack centers on Django 4.2, PostgreSQL, scikit-learn, and Facebook Prophet. Data collection combines six real-world and synthetic datasets totaling over 140,000 records across five business domains. The following

chapter presents the detailed system analysis and design, including use case diagrams, class diagrams, and database schema design.

Bibliography

Al-Kraiji, A. and Al-Saedi, K. (2020). Challenges of ERP localization in Arabic-speaking countries: A systematic review. In *Proceedings of the International Conference on Information Systems*, pages 1–15.

Al-Majali, D. (2016). Antecedents of IT-business strategic alignment factors and its impact on organizational performance. *International Review of Management and Business Research*, 5(4):1290–1301.

Bezemer, C.-P. and Zaidman, A. (2010). Multi-tenant SaaS applications: Maintenance dream or nightmare? *Proceedings of the Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution*, pages 88–92.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Chong, F. and Carraro, G. (2006). Architecture strategies for catching the long tail. *Microsoft Developer Network (MSDN)*.

Davenport, T. H. and Ronanki, R. (2018). *Artificial Intelligence for the Real World*, volume 96.

Fildes, R., Ma, S., and Kolassa, S. (2022). Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4):1283–1318.

Grand View Research (2023). Enterprise Resource Planning Market Size, Share & Trends Analysis Report. Technical report, Grand View Research. Report ID: GVR-1-68038-955-0.

IBM (2017). IBM HR Analytics Employee Attrition & Performance Dataset. Kaggle. https://www.kaggle.com/pavansubhasht/ibm-hr-analytics-attrition-dataset.

Jacobs, F. R. and Weston, F. C. (2007). Enterprise resource planning (ERP) — a brief history. *Journal of Operations Management*, 25(2):357–363.

Krafzig, D., Banke, K., and Slama, D. (2005). *Enterprise SOA: Service-Oriented Architecture Best Practices*. Prentice Hall, Upper Saddle River, NJ.

Lee, J., Siau, K., and Hong, S. (2003). Enterprise integration with ERP and EAI. *Communications of the ACM*, 46(2):54–60.

Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422.

Monk, E. and Wagner, B. (2013). *Concepts in Enterprise Resource Planning*. Cengage Learning, Boston, MA, 4th edition.

Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA.

Olist (2018). Brazilian e-commerce public dataset by olist. Kaggle. https://www.kaggle.com/olistbr/brazilian-ecommerce.

Panorama Consulting Group (2023). 2023 ERP Report: Clash of the Titans. Technical report, Panorama Consulting Group, Denver, CO.

Provost, F. and Fawcett, T. (2013). *Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking*. O'Reilly Media, Sebastopol, CA.

Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide: The Definitive Guide to Scrum — The Rules of the Game*. Scrum.org.

Shewhart, W. A. (1931). *Economic Control of Quality of Manufactured Product*. D. Van Nostrand Company, New York.

Society for Human Resource Management (2022). 2022 Employee Benefits Survey. Technical report, SHRM, Alexandria, VA.

Taylor, S. J. and Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1):37–45.